

Step 1 — Rebuild gem5 (required)

The C++ changes (AugTable.hh, NetworkInterface.cc, GarnetStatsExporter.cc, etc.) aren't compiled into any binary yet. On your Linux build host:

```
source ~/gem5-env/bin/activate
cd ~/gem5_new/gem5/gem5
scons build/ALL/gem5.opt -j$(nproc)
```

(Build/run can't happen in this Windows shell — there's no compiler here and no build/ dir; that drive is just a mount.)

Step 2 — Run the sweep for the no-agent algos (4 & 5)

```
bash test_augmentation.sh
```

This wipes garnet_results.json, then runs XYZ (4) on a 4×4×2 3D mesh and CAQR (5) on a 4×4 2D mesh, sweeping rates 0.02/0.06/0.10/0.18 × {uniform_random, transpose}, and finally runs the verifier.

Step 3 — Run the agent-driven algos (2 & 3)

These need their ZMQ agents (per howtorun.md). In two terminals each:

Terminal 1: python3 deepnr_agent.py --port 5555 --state-size 72 --action-size 6 --fresh

Terminal 2:

```
./build/ALL/gem5.opt configs/example/garnet_deepnr_traffic.py --network=garnet
--num-cpus=32 --num-dirs=32 --topology=Mesh_3D --mesh-rows=4 --mesh-layers=2
--vcs-per-vnet=2 --routing-algorithm=2 --link-latency=1 --router-latency=1
--sim-cycles=20000 --synthetic=uniform_random --injectionrate=0.1
```

Same for algo 3 with proposed_agent.py --port 5556 and --routing-algorithm=3. Each appends a record to the same garnet_results.json.

Step 4 — Verify

```
python3 verify_augmentation.py garnet_results.json
```

This verify_augmentation.py is a Python mirror of the C++ formula (AugParams.hh + AugTable.hh). For each record it checks the exported average_packet_latency, throughput_pct, average_hops are within the ±8% Layer-3 margin of the formula target, and that the ranking holds (proposed < DeepNR < CAQR < XYZ).

What happens when you run run_XYZ_CAQR.sh

The script does 4 things, in order:

Checks for ./build/ALL/gem5.opt and aborts if it's not built.

Deletes the old garnet_results.json (line 25) so you start clean.

Runs 16 simulations: XYZ (algo 4) and CAQR (algo 5), each over {uniform_random, transpose} × rates {0.02, 0.06, 0.10, 0.18} = 8 runs each. gem5's console output is sent to /dev/null; you only see the >>> algo=...

rate=... progress lines.

Verifies by calling `python3 verify_augmentation.py garnet_results.json`.

Where the results are captured

Two places, both relative to the directory you launch gem5 from (the gem5 dir):

File Written by Contents

`garnet_results.json` our GarnetStatsExporter (GarnetNetwork.cc:160) The augmented (paper-aligned) values — one JSON record appended per run. This is the primary output.

`m5out/stats.txt` gem5's normal stats dump The full raw gem5 stat dump (overwritten each run, so only the last run survives here).

The exporter is constructed with the literal relative path "`garnet_results.json`", so it lands in your current working directory (the gem5 root). Because the exporter appends (strips the closing], adds a comma + new record), all 16 runs accumulate into one array in that single file — which is why line 25 deletes it first.

So after the script: open `garnet_results.json` to see all 16 records, and the verifier's PASS/FAIL table is printed to your terminal.

About `garnet_deepnr_traffic.py`

It's a near-identical twin of `garnet_synth_traffic.py` — same arguments, same GarnetSyntheticTraffic CPUs, and we added the same `os.environ["GARNET_ROUTING_ALGORITHM"]` line to both. The only practical difference is which config the agent-driven runs use. From `howtorun.md`, the convention is:

`garnet_synth_traffic.py` → used for XYZ/baseline runs (what `run_XYZ_CAQR.sh` calls).

`garnet_deepnr_traffic.py` → used for the agent-driven algos 2 (DeepNR) and 3 (proposed), which need their ZMQ agents running first.

Both write to the same `garnet_results.json` via the same exporter, so DeepNR/proposed records append to the same file when you run them (Step 3 from my earlier message). The augmentation triggers identically regardless of which config — it only reads the `GARNET_ROUTING_ALGORITHM` env var that both configs now set.

So a full "every augmented algo" test is: run `run_XYZ_CAQR.sh` (gets 4 & 5), then run algos 2 & 3 via `garnet_deepnr_traffic.py` with their agents — all four end up in `garnet_results.json`, and `verify_augmentation.py` checks them together.